

Politechnika Świętokrzyska Wydział Elektrotechniki Automatyki i Informatyki		
Bazy Danych – Laboratorium		
Numer grupy 2ID14A	Temat projektu: System obsługi bazy danych dla kompleksu hotelowego	Informatyka - II rok, Rok akademicki - 2025/2026
Wykonali: Isaiev Yehor Khoprov Yakiv Romanenko Mykyta		

1. Opis projektu

Celem projektu było stworzenie aplikacji okienkowej do zarządzania hotelem. System pozwala na obsługę bazy danych PostgreSQL, w tym zarządzanie pokojami, klientami oraz procesem rezerwacji. Główne zadania aplikacji to automatyzacja pracy recepcji oraz generowanie statystyk finansowych.

2. Instrukcja obsługi

2.1 Wymagania i instalacja

Aplikacja zbudowana w Javie z wykorzystaniem Maven.

- **Java:** wersja 21.
- **Baza danych:** PostgreSQL.
- **Biblioteki:** JavaFX (UI), BCrypt (haszowanie haseł), Log4j2 (logi).

2.2 Uruchomienie

1. Skonfiguruj bazę danych PostgreSQL.
2. Zbuduj projekt: `mvn clean install`.
3. Uruchom serwer: Plik (`src\main\java\com\hotel\server\ServerApp.java`)
4. Uruchom klienta: Plik (`src\main\java\com\hotel\Launcher.java`)

3. Specyfikacja techniczna

Aplikacja dzieli się na moduły zarządzające konkretnymi obszarami danych:

- **Klasa DatabaseHandler:** Odpowiada za logikę dostępu do danych. Realizuje wszystkie zapytania SQL do bazy PostgreSQL, w tym operacje CRUD dla pokoi, klientów i rezerwacji, a także zaawansowaną agregację danych statystycznych.
- **Klasa AdminController:** Główny kontroler warstwy prezentacji dla panelu administratora. Zarządza obiektami TableView, implementuje filtrowanie danych przy użyciu Java Streams oraz obsługuje zdarzenia interfejsu, komunikując się z serwerem poprzez NetworkClient.
- **Klasa NetworkClient:** Warstwa komunikacyjna klienta. Odpowiada za otwieranie gniazd (Sockets), serializację obiektów typu Request i odbieranie odpowiedzi Response ze strumienia wejściowego.
- **Klasa ClientHandler:** Klasa serwerowa implementująca interfejs Runnable. Odpowiada za wielowątkową obsługę połączeń przychodzących, deszyfrowanie żądań klientów i mapowanie ich na konkretne wywołania metod w DatabaseHandler.
- **Klasa DatabaseConfig:** Moduł konfiguracyjny serwera. Wykorzystuje klasę Properties do dynamicznego ładowania parametrów połączenia z bazą danych (URL, login, hasło) z zewnętrznego pliku database.properties.

4. Architektura aplikacji

Projekt został zaprojektowany w modelu trójwarstwowym, co pozwala na całkowite odizolowanie kodu odpowiedzialnego za wygląd od logiki przetwarzania danych i samej bazy.

4.1 Warstwa prezentacji

Odpowiada za wszystko, co widzi użytkownik oraz za przechwytywanie jego akcji.

- **Technologia:** Wykorzystano bibliotekę JavaFX 21. Wygląd okien zdefiniowany jest w plikach .fxml, a ich stylizacja odbywa się poprzez zewnętrzne arkusze stylów CSS, co pozwala na szybką zmianę designu bez ingerencji w kod Java.
- **Kontrolery:** Każdy widok posiada swój kontroler, który zarządza komponentami takimi jak TableView (do wyświetlania danych), TextField (do wyszukiwania) czy Button.
- **Bindowanie danych:** Wykorzystano mechanizm, który automatycznie łączy pola obiektów (np. Room, Client) z kolumnami w tabelach interfejsu.

4.2 Warstwa logiki

Stanowi „mózg” aplikacji i pośredniczy w komunikacji między użytkownikiem a bazą danych.

- **Komunikacja Klient-Serwer:** Zaimplementowano architekturę sieciową opartą na gniazdach (Sockets). Klient wysyła obiekt Request, a serwer przesyła z powrotem obiekt Response.
- **Obsługa żądań:** Klasa ta działa w oddzielnym wątku dla każdego połączenia, rozpoznaje typ zapytania i wywołuje odpowiednie metody.

- **Autoryzacja i role:** System sprawdza uprawnienia użytkownika. Na podstawie roli zwróconej z bazy (ADMIN lub CLIENT), aplikacja decyduje, jakie widoki i funkcje zostaną udostępnione (np. tylko administrator widzi panel zarządzania pokojami).
- **Wstępna obróbka danych:** Przed zapisem do bazy system sprawdza poprawność danych, np. czy cena jest liczbą dodatnią oraz hashuje hasła za pomocą algorytmu BCrypt dla zapewnienia bezpieczeństwa.

4.3 Warstwa dostępu do danych

Zajmuje się bezpośrednimi operacjami na systemie zarządzania bazą danych PostgreSQL.

- **Klasa DatabaseHandler:** Jest to centralny punkt dostępu do bazy. Wszystkie instrukcje SQL (SELECT, INSERT, UPDATE, DELETE) są tu zdefiniowane.
- **Zarządzanie połączeniem:** Konfiguracja bazy (URL, port, nazwa użytkownika) nie jest wpisana na stałe w kodzie, lecz ładowana z pliku database.properties. Pozwala to na łatwe przeniesienie aplikacji na inny serwer bez zmiany kodu źródłowego.
- **Obsługa transakcji:** Kluczowe operacje, jak np. rejestracja użytkownika, są objęte transakcjami. Jeśli zapis w jednej tabeli (np. users) się powiedzie, a w drugiej (np. clients) zawiedzie, system wykonuje rollback, co zapobiega powstawaniu błędnych lub niepełnych danych w systemie.

5. Szczegółowa implementacja instrukcji SQL

W aplikacji „Hotel System” wykorzystano pełen zestaw instrukcji SQL do zarządzania relacyjną bazą danych PostgreSQL. Poniżej przedstawiono opisy i fragmenty kodu dla każdej z nich.

5.1 SELECT (Pobieranie danych)

Instrukcje SELECT są fundamentem wyświetlania informacji w interfejsie użytkownika. Wykorzystano zarówno proste zapytania, jak i zaawansowane złączenia (JOIN) oraz podzapytania agregujące.

- **Zastosowanie:** Pobieranie listy rezerwacji wraz z powiązanymi usługami dodatkowymi.
- **Kod:**

```
SELECT b.id, b.client_id, b.room_id, b.check_in_date, b.check_out_date,
       b.total_price, b.status, u.email, r.room_number, h.name as hotel_name,
       (SELECT array_agg(amenity_id) FROM booking_amenities WHERE booking_id = b.id)
as amenity_ids
FROM bookings b
JOIN clients c ON b.client_id = c.id
JOIN users u ON c.user_id = u.id
JOIN rooms r ON b.room_id = r.id
```

```
JOIN hotels h ON r.hotel_id = h.id  
ORDER BY b.id DESC;
```

5.2 INSERT (Dodawanie danych)

Instrukcje INSERT służą do rejestracji użytkowników, dodawania pokoi oraz tworzenia nowych rezerwacji. Często wykorzystują klauzulę RETURNING id do dalszych operacji.

- **Zastosowanie:** Rejestracja nowego użytkownika w tabeli systemowej.
- **Kod:**

```
INSERT INTO users (email, password, role)  
VALUES (p_email, p_password, 'CLIENT')  
RETURNING id INTO new_user_id;
```

5.3 UPDATE (Modyfikacja danych)

Służą do aktualizacji istniejących rekordów, takich jak zmiana danych klienta czy parametrów pokoju.

- **Zastosowanie:** Edycja parametrów pokoju przez administratora.
- **Kod:**

```
UPDATE rooms SET room_number=?, type=?, price_per_night=?, description=?  
WHERE id=?
```

5.4 DELETE (Usuwanie danych)

Wykorzystywane do usuwania nieaktualnych rezerwacji lub rekordów bazy danych.

- **Zastosowanie:** Usunięcie rezerwacji o konkretnym identyfikatorze.
- **Kod:**

```
DELETE FROM bookings  
WHERE id = ?
```

5.5 CREATE i DROP (Definiowanie struktury DDL)

Instrukcje te zarządzają strukturą tabel i typów danych w systemie.

- **Zastosowanie:** Tworzenie tabeli hoteli oraz usuwanie niepotrzebnych powiązań usług.
- **Kod (CREATE):**

```
CREATE TABLE hotels (  
  id SERIAL PRIMARY KEY,  
  name VARCHAR(100) NOT NULL,  
  city VARCHAR(50) NOT NULL,  
  address VARCHAR(200) NOT NULL,  
  rating INT DEFAULT 3 CHECK (rating BETWEEN 1 AND 5)  
);
```

- **Kod (DROP):**

```
DROP TABLE IF EXISTS booking_amenities CASCADE;
```

5.6 PROCEDURA (Logika proceduralna PL/pgSQL)

Procedury składowane pozwalają na wykonywanie złożonej logiki bezpośrednio po stronie bazy danych.

- **Zastosowanie:** Procedura rejestracji konta klienta w systemie.
- **Kod:**

```
CREATE OR REPLACE PROCEDURE register_client(  
  p_email VARCHAR,  
  p_password VARCHAR,  
  p_first_name VARCHAR,  
  p_last_name VARCHAR,  
  p_phone VARCHAR  
)  
LANGUAGE plpgsql  
AS $$  
DECLARE  
  new_user_id INT;  
BEGIN
```

```
INSERT INTO users (email, password, role)
VALUES (p_email, p_password, 'CLIENT')
RETURNING id INTO new_user_id;

INSERT INTO clients (user_id, first_name, last_name, phone)
VALUES (new_user_id, p_first_name, p_last_name, p_phone);

END;
$$;
```

5.7 FUNKCJA (Przetwarzanie danych)

Wykorzystanie wbudowanych i zdefiniowanych funkcji SQL do operacji na danych i agregacji.

- **Zastosowanie:** Wyliczanie statystyk miesięcznych przychodów za pomocą funkcji EXTRACT.
- **Kod:**

```
CREATE OR REPLACE FUNCTION get_monthly_income(p_month INT, p_year INT)
RETURNS DECIMAL(10, 2)
LANGUAGE plpgsql
AS $$
DECLARE
    income DECIMAL(10, 2);
BEGIN
    SELECT COALESCE(SUM(total_price), 0)
    INTO income
    FROM bookings
    WHERE EXTRACT(MONTH FROM created_at) = p_month
    AND EXTRACT(YEAR FROM created_at) = p_year
    AND status != 'CANCELLED';

    RETURN income;
END;
$$;
```

Dzięki zastosowaniu powyższych instrukcji, aplikacja zapewnia pełną integralność danych oraz realizuje wszystkie wymagania funkcjonalne typowe dla systemów obsługi przedsiębiorstw.

6. Instrukcja użytkownika

Akcja	Opis działania
Logowanie	Autoryzacja użytkownika na podstawie adresu email i hasła. System weryfikuje rolę (Admin/Client) i przekierowuje do odpowiedniego widoku.
Rejestracja	Tworzenie nowego konta klienta. System przeprowadza walidację danych, haszuje hasło algorytmem BCrypt i tworzy rekordy w tabelach users oraz clients w ramach jednej transakcji.
Dashboard (Statystyki)	Ekran powitalny administratora wyświetlający utarg z bieżącego miesiąca, liczbę dzisiejszych rezerwacji oraz listę 15 ostatnich aktywności w systemie.
Zarządzanie Pokojami	Przeglądanie listy pokoi, dodawanie nowych jednostek, edycja parametrów (cena, typ, opis) oraz usuwanie pokoi z bazy danych.
Zarządzanie Klientami	Moduł pozwalający administratorowi na pełną edycję danych gości (imię, nazwisko, telefon, email) lub usuwanie nieaktywnych kont.
Obsługa Rezerwacji	Tworzenie nowych rezerwacji poprzez przypisanie klienta do pokoju w wybranym terminie. Możliwość edycji terminów, zmiany statusu (np. na CANCELLED) oraz usuwania wpisów.
Usługi Dodatkowe (Amenities)	Wybór opcjonalnych usług (np. śniadanie, basen) podczas tworzenia lub edycji rezerwacji. System automatycznie uwzględnia je w cenie całkowitej.

Filtrowanie i Szukanie	Dynamiczne przeszukiwanie tabel w czasie rzeczywistym. Możliwość filtrowania pokoi po nazwie hotelu, klientów po emailu oraz rezerwacji po statusie lub numerze pokoju.
Zarządzanie Profilem	Moduł dla klienta i pracownika pozwalający na podgląd własnych danych oraz zmianę informacji kontaktowych.
Wylogowanie	Bezpieczne zakończenie bieżącej sesji użytkownika i powrót do ekranu logowania.

7. Spełnienie wymagań projektowych

W ramach projektu zrealizowano wszystkie kluczowe wymagania zawarte w specyfikacji zadania. Poniżej przedstawiono szczegóły implementacyjne:

7.1. Zaawansowane zapytania SQL i agregacja danych

Aplikacja wykorzystuje złożone instrukcje SELECT z funkcjami agregującymi oraz funkcjami operującymi na danych do generowania raportów finansowych.

Przykład (SQL w DatabaseHandler):

```
SELECT b.created_at, u.email, r.room_number, b.status " +
    "FROM bookings b " +
    "JOIN clients c ON b.client_id = c.id " +
    "JOIN users u ON c.user_id = u.id " +
    "JOIN rooms r ON b.room_id = r.id " +
    "ORDER BY b.created_at DESC LIMIT 15
```

7.2. Obsługa transakcji i integralność danych

W celu zapewnienia atomowości operacji (np. podczas rejestracji, gdzie rekordy muszą zostać dodane do dwóch powiązanych tabel), zaimplementowano mechanizm transakcji.

Przykład (Transakcja w registerUser):

```
String sql = "CALL register_client(?, ?, ?, ?, ?)";

try (Connection conn = getConnection());
```



```

        java.sql.CallableStatement cstmt = conn.prepareCall(sql)) {

String hashedPassword = PasswordHasher.hashPassword(password);

cstmt.setString(1, email);
cstmt.setString(2, hashedPassword);
cstmt.setString(3, firstName);
cstmt.setString(4, lastName);
cstmt.setString(5, phone);

cstmt.execute();

logger.info("Rejestracja zakończona sukcesem (via procedure): {}", email);
return true;

} catch (SQLException e) {
    logger.error("Błąd procedury rejestracji {}: {}", email, e.getMessage());
    return false;
}

```

7.3. Modularność i rozdzielenie warstw

Projekt posiada wyraźną separację warstw:

- **Prezentacja:** Pliki FXML i klasy Controller.
- **Logika biznesowa:** Serwer i system żądań.
- **Dostęp do danych:** SQL zlokalizowane wyłącznie w DatabaseHandler.java.

7.4. Ergonomia i filtrowanie danych

Interfejs użytkownika umożliwia dynamiczne przeszukiwanie zasobów bez konieczności przeładowywania całej listy, co zwiększa wydajność pracy.

Przykład (Filtrowanie pokoi):

```

@FXML
protected void onSearchRoom() {
    String query = searchRoomField.getText().toLowerCase().trim();
    if (query.isEmpty()) {
        roomsTable.setItems(masterRoomData);
    } else {
        ObservableList<Room> filteredList = masterRoomData.stream()
            .filter(room -> room.getHotelName() != null &&
                room.getHotelName().toLowerCase().contains(query))
            .collect(Collectors.toCollection(FXCollections::observableArrayList));
        roomsTable.setItems(filteredList);
    }
}

```

7.5. Bezpieczeństwo danych

Hasła użytkowników nie są przechowywane w bazie danych w formie jawnej. Zastosowano bibliotekę BCrypt do generowania soli i haszowania haseł przed zapisem do tabeli users.

Tabela amenities w bazie danych PostgreSQL:

	id [PK] integer	name character varying (100)	price numeric (10,2)
1	1	Śniadanie (Breakfast)	40.00
2	2	Parking Podziemny	25.00
3	3	Dostęp do SPA	100.00
4	4	Późne wymeldowanie (Late Check-o...	50.00
5	5	Łóżeczko dziecięce	0.00

Tabela bookings w bazie danych PostgreSQL:

	id [PK] integer	client_id integer	room_id integer	check_in_date date	check_out_date date	total_price numeric (10,2)	status booking_status	created_at timestamp without time zone
1	1	4	2	2026-01-20	2026-01-25	2640.00	CONFIRMED	2026-01-19 23:13:56.319521
2	2	2	1	2026-02-03	2026-02-08	1890.00	PENDING	2026-01-19 23:14:22.470829
3	3	3	4	2027-01-05	2028-01-29	466875.00	CONFIRMED	2026-01-19 23:14:41.789669
4	4	5	3	2026-02-05	2026-03-06	9355.00	CONFIRMED	2026-01-19 23:15:49.879993

Tabela users w bazie danych PostgreSQL:

	id [PK] integer	email character varying (100)	password character varying (200)	role user_role	created_at timestamp without time zone
1	1	admin@hotel.com	\$2a\$12\$sbw59MGJ32aKCJ4vqMM/ZO9fNR.siatNldmc5yFoxwmQCXJLnfhbq	ADMIN	2026-01-19 21:35:18.038231
2	3	albert@email.com	\$2a\$12\$PtFyCebW4/fDC13P6sLPB./0BRqRsUWkthAt036eeEDY52J1nNgi.	CLIENT	2026-01-19 23:10:28.645849
3	4	cristiano@email.com	\$2a\$12\$awJzp7JZ9F6Ka522FYkntOSXAFUJeXIdJG7YdXlzfJKZjp0umr5G	CLIENT	2026-01-19 23:11:47.690875
4	5	zoro@email.com	\$2a\$12\$tsaoVXhmY.hBoeoOckpWDurm.iftEpBgwKopOP.ZqqZV9CnqtGWoy	CLIENT	2026-01-19 23:12:29.805129
5	6	pitt@email.com	\$2a\$12\$1Cs2Ao001oZCfWo2CZHc3.UiEbHyfl4eBhIVFluq/uulhgHH//c2W	CLIENT	2026-01-19 23:13:38.82572

Tabela clients w bazie danych PostgreSQL:

	id [PK] integer	user_id integer	first_name character varying (100)	last_name character varying (100)	phone character varying (20)
1	2	3	Albert	Einstein	+123123123123
2	3	4	Cristiano	Ronaldo	+380123123123
3	4	5	Zoro	Roronoa	+48575575575
4	5	6	Pitt	Brad	+23454631234

Tabela hotels w bazie danych PostgreSQL:

	id [PK] integer	name character varying (100)	city character varying (50)	address character varying (200)	rating integer
1	1	Hotel Warsaw Central	Warszawa	Aleje Jerozolimskie 1	3
2	2	Krakow Old Town Inn	Kraków	Rynek Główny 5	3

Tabela rooms w bazie danych PostgreSQL:

	id [PK] integer	hotel_id integer	room_number character varying (10)	type character varying (50)	capacity integer	price_per_night numeric (10,2)	floor integer	status room_status	description text
1	1	1	101	Standard	1	350.00	1	FREE	Przytulny pokój z widokiem na patio
2	2	1	102	Double	2	500.00	1	OCCUPIED	Przestronny pokój z dużym łóżkiem
3	3	2	101	Standard	1	320.00	1	FREE	Cichy pokój w stylu vintage
4	4	2	201	Suite	4	1200.00	2	CLEANING	Luksusowy apartament z dwoma sypialni...

8. Podsumowanie i wnioski

8.1 Realizacja celów projektu

Projekt „Hotel System” został zrealizowany w pełnej zgodności z dokumentacją techniczną. Aplikacja w 100% wypełnia kryteria punktowe poprzez:

- **Pełną obsługę operacji SQL:** Zaimplementowano instrukcje SELECT, INSERT, UPDATE oraz DELETE dla wszystkich kluczowych modułów (Pokoje, Klienci, Rezerwacje).
- **Wykorzystanie logiki proceduralnej:** W projekcie użyto zaawansowanych mechanizmów PostgreSQL, takich jak funkcje agregujące oraz anonimowe bloki proceduralne do zarządzania kontami administracyjnymi.
- **Bezpieczeństwo i stabilność:** System jest odporny na błędy dzięki rozbudowanej obsłudze wyjątków SQLException oraz bezpiecznemu hashowaniu haseł biblioteką BCrypt.

8.2 Interfejs użytkownika i ergonomia

Warstwa prezentacji została zaprojektowana z myślą o maksymalnej użyteczności dla pracownika hotelu.

- **Intuicyjność:** Zastosowanie JavaFX pozwoliło na stworzenie czytelnego układu z bocznym menu nawigacyjnym, co umożliwia szybkie przełączanie się między widokami bez utraty kontekstu pracy.
- **Praca z danymi:** Wszystkie tabele posiadają mechanizmy dynamicznego filtrowania, co znacznie skraca czas wyszukiwania konkretnego gościa czy rezerwacji w dużej bazie danych.
- **Komunikacja z użytkownikiem:** System informuje o wyniku każdej operacji (dodanie, edycja, usunięcie) za pomocą okien dialogowych i alertów, co minimalizuje ryzyko pomyłek.

8.3 Potencjał wdrożeniowy i perspektywy rozwoju

Stworzona aplikacja jest solidnym fundamentem pod system klasy Hotel Management System.

- **Gotowość do pracy:** Architektura klient-serwer oparta na gniazdach (Sockets) pozwala na jednoczesną pracę wielu stanowisk recepcyjnych podłączonych do jednego serwera bazy danych.
- **Skalowalność:** Dzięki modułowej strukturze kodu (separacja warstw), system można łatwo rozbudować o nowe funkcjonalności, takie jak:
 - Integracja z systemami płatności online.
 - Moduł generowania faktur do plików PDF.
 - System zarządzania statusem sprzątania pokoi dla personelu sprzątającego.

Wnioski

Projekt udowadnia, że połączenie relacyjnej bazy danych PostgreSQL z nowoczesnym interfejsem JavaFX pozwala na budowę profesjonalnych narzędzi do kontroli i zarządzania złożonymi systemami hotelowymi.